TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

Unidad 5
Redes

Fetch · Axios · TanStack Query

Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Expo SDK 57 · React Native 0.86 · React 19.2

UNIDAD 5

Qué vamos a ver

Objetivos

- Pedir y enviar datos a un servidor.
- Manejar carga, error y cancelación como parte del diseño.
- Centralizar la configuración de la API.
- Dejar de escribir a mano la sincronización con el servidor.

Partes

- Hablar con un servidor
- Axios
- TanStack Query

Persistencia, estado global y conectividad van en el deck siguiente.

UNIDAD 5 · PARTE 1

Hablar con un servidor

Por qué en móvil es distinto

En el navegador de una computadora

- La conexión casi siempre está.
- Cambiar de red es raro.
- Si falla, el usuario recarga.

En un teléfono

- Se entra y se sale del ascensor, del subte, del campo.
- Wi-Fi → datos móviles a mitad de un request.
- «Conectado» no significa que haya internet: el portal cautivo del bar te contesta a todo.
- No hay botón de recargar.

Consecuencia práctica: **el estado de error y el de carga no son excepciones, son parte del diseño de la pantalla**. Una pantalla que solo contempla el caso feliz está a medio hacer.

Fetch, lo provee React Native

```
async function traerProductos() {
  const respuesta = await fetch('https://api.ejemplo.com/productos', {
    method: 'GET',
    headers: { 'Content-Type': 'application/json' },
  });

  const datos = await respuesta.json();
  return datos;
}
```

Ya está disponible

React Native lo expone como global, igual que el navegador. No se instala nada.

Basado en promesas

Funciona con `async/await` sin vueltas.

En contra

Sin JSON automático, sin tiempo de espera, sin reintentos, y con una trampa grande.

La trampa: un 404 no es un error

LO QUE CASI TODOS ESCRIBEN

```
try {
  const r = await fetch(url);
  const datos = await r.json();
  mostrar(datos);          // ← con un 404 llega acá
} catch (e) {
  mostrarError(e);         // ← nunca entra
}
```

LO QUE HAY QUE ESCRIBIR

```
try {
  const r = await fetch(url);

  if (!r.ok) {             // ← el chequeo que falta
    throw new Error(`El servidor respondió ${r.status}`);
  }

  mostrar(await r.json());
} catch (e) {
  mostrarError(e);
}
```

`fetch` solo rechaza la promesa si **no llegó a hablar con el servidor** —sin red, DNS caído, TLS inválido—. Un 404 o un 500 son respuestas exitosas desde su punto de vista: **chequeá `response.ok` siempre**.

Los tres estados de cualquier request



Cargando | Error — con salida | Dato

> Las tres se diseñan. La del medio es la que siempre falta, y es la única que el usuario va a ver justo cuando más lo necesita.

Cancelar el request

```
useEffect(() => {
  const control = new AbortController();

  (async () => {
    try {
      const r = await fetch(url, { signal: control.signal });
      if (!r.ok) throw new Error(`Error ${r.status}`);
      setDatos(await r.json());
    } catch (e) {
      if (e.name !== 'AbortError') setError(e);   // cancelar no es fallar
    }
  })();

  return () => control.abort();
}, [url]);
```

Sin esto: el usuario sale de la pantalla, la respuesta llega igual y React avisa que actualizaste un componente desmontado. En una lista donde se escribe para buscar, además se pisan las respuestas y gana la más lenta.

UNIDAD 5 · PARTE 2

Axios

Lo mismo, con lo repetido ya resuelto.

Qué te ahorra

JSON automático

`response.data` ya viene parseado. Sin `await r.json()`.

Errores de verdad

Un 404 o un 500 **sí** rechazan la promesa. Se acabó el `response.ok`.

Tiempo de espera

`timeout` por configuración. Fetch no lo tiene.

Interceptores

Un lugar único donde tocar todas las peticiones y todas las respuestas.

> También cancela, igual que Fetch y con la misma `AbortController`: no es una API distinta, es la misma del estándar.

Un cliente para toda la app

```
// api/cliente.ts — se escribe una vez
import axios from 'axios';

export const api = axios.create({
  baseURL: process.env.EXPO_PUBLIC_API_URL,   // https://api.ejemplo.com
  timeout: 10_000,
  headers: { 'Content-Type': 'application/json' },
});
```

```
// en cualquier pantalla
import { api } from '@/api/cliente';

const { data } = await api.get('/productos');
await api.post('/productos', { nombre, precio });
```

> La URL del servidor aparece **una sola vez** en todo el proyecto. Cuando pases de tu máquina a producción, cambiás una variable de entorno y no veinte archivos.

Interceptores: el token, en un solo lugar

```
// Antes de cada petición: adjuntar el token.
api.interceptors.request.use(async (config) => {
  const token = await leerToken();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Después de cada respuesta: reaccionar a un 401 en un solo lado.
api.interceptors.response.use(
  (respuesta) => respuesta,
  (error) => {
    if (error.response?.status === 401) cerrarSesion();
    return Promise.reject(error);
  },
);
```

> Sin interceptores, ese `if (401)` queda copiado en cada pantalla que llama a la API. El token va a `expo-secure-store`, no a un archivo de texto — lo vemos en la Unidad 6.

El error de Axios tiene tres formas

```
try {
  const { data } = await api.get('/productos');
} catch (e) {
  if (e.response) {
    // Llegó al servidor y contestó mal. Es el caso más informativo.
    console.log(e.response.status, e.response.data);
  } else if (e.request) {
    // Salió el request y nunca volvió: sin red, servidor caído, timeout.
    mostrar('No pudimos conectarnos. Revisá tu conexión.');
  } else {
    // Ni salió: error de configuración del request. Es un bug tuyo.
    console.error(e.message);
  }
}
```

> Esa distinción es la que te deja escribir un mensaje útil. «Algo salió mal» no le sirve a nadie; «no hay conexión» y «ese producto no existe» sí.

¿Fetch o Axios?

FETCH ALCANZA SI…

- Son dos o tres requests sueltos.
- No hay autenticación.
- Querés cero dependencias.

AXIOS CONVIENE SI…

- Hay una API propia con muchos endpoints.
- Hay token y sesión que expira.
- Querés el mismo manejo de errores en toda la app.

Ninguno de los dos resuelve el problema que viene ahora: **qué hacer con el dato una vez que llegó**. Dónde se guarda, cuándo está viejo, quién lo vuelve a pedir.

UNIDAD 5 · PARTE 3

TanStack Query

El dato del servidor no es estado de tu app: es una copia.

Dos clases de estado

ESTADO DEL CLIENTE

- El texto de un formulario, si el modal está abierto, el tema oscuro.
- **Vos sos el dueño.** Es la verdad.
- Sincrónico y siempre disponible.

`useState` , Context, Zustand.

ESTADO DEL SERVIDOR

- La lista de productos, el perfil, los pedidos.
- **El dueño es otro.** Vos tenés una copia que envejece.
- Asincrónico, puede fallar, puede quedar viejo sin avisarte.

TanStack Query.

Meter el estado del servidor en un `useState` es el error de diseño más común. Funciona, y después hay que escribir a mano caché, reintentos, invalidación y refresco al volver a la app.

Lo mismo, en dos versiones

A MANO

```
const [datos, setDatos] = useState(null);
const [cargando, setCargando] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  const c = new AbortController();
  setCargando(true);
  api.get('/productos', { signal: c.signal })
    .then(r => setDatos(r.data))
    .catch(e => { if (e.name !== 'CanceledError') setError(e); })
    .finally(() => setCargando(false));
  return () => c.abort();
}, []);
```

CON TANSTACK QUERY

```
const { data, isPending, error } = useQuery({
  queryKey: ['productos'],
  queryFn: () => api.get('/productos')
                    .then(r => r.data),
});
```

Y encima trae caché compartida, reintentos, refresco en segundo plano y deduplicación de requests simultáneos.

> No es solo que sea más corto: la versión de la izquierda **no comparte** nada. Dos pantallas que muestran los productos hacen dos requests y mantienen dos copias que se desincronizan.

Las opciones que importan

`queryKey`

La identidad en la caché. Todo lo que cambie el resultado va adentro: cambiar la clave es pedir otra cosa.

```
queryKey: ['productos', categoria, pagina]
```

`queryFn`

Devuelve el dato o **tira un error**. Con Axios ya viene; con Fetch hay que tirarlo a mano.

```
queryFn: () => api.get('/productos').then(r => r.data)
```

`staleTime`

Cuánto vale la copia antes de salir a chequear. Por defecto **0**: todo está viejo enseguida.

```
staleTime: 60_000        // un minuto
```

`gcTime` · `retry`

Cuánto sobrevive en memoria lo que ya nadie mira, y cuántos reintentos ante un fallo.

```
gcTime: 5 * 60_000,      // por defecto
retry: 2                 // por defecto 3
```

> `staleTime` y `gcTime` se confunden siempre. **Stale** = «lo muestro igual, pero salgo a chequear». **gc** = «lo borro de memoria».

Los dos hooks

`useQuery` — LEER

- Corre **solo**, cuando el componente se monta.
- Guarda lo que trae en la caché, bajo su `queryKey`, y lo comparte con toda la app.
- Repetirlo no tiene consecuencias: por eso puede reintentar y refrescar en segundo plano sin preguntarte.

Lee. Es el `GET`.

`useMutation` — ESCRIBIR

- No corre solo: lo disparás vos, con `.mutate(datos)`.
- No se cachea. Lo que devuelve no es un dato para mostrar, es la confirmación de un efecto.
- Repetirlo **sí** tiene consecuencias —crea dos veces—, así que no reintenta por su cuenta.

Escribe. Es el `POST`, `PUT` o `DELETE`.

Escribir: `useMutation`

```
import { useMutation, useQueryClient } from '@tanstack/react-query';

const queryClient = useQueryClient();

const crear = useMutation({
  mutationFn: (nuevo) => api.post('/productos', nuevo).then(r => r.data),

  onSuccess: () => queryClient.invalidateQueries({ queryKey: ['productos'] }),

  onError: (e) => mostrarError(e),
});

<Button title="Guardar" onPress={() => crear.mutate({ nombre, precio })} />
```

Así se enganchan los dos hooks: una mutación que sale bien deja la caché vieja, y por eso su último paso es **invalidar la query**, para que se vuelva a pedir sola.

`invalidateQueries` lleva un **objeto**: `{ queryKey: ['productos'] }`.

Actualización optimista

```
const marcar = useMutation({
  mutationFn: (id) => api.patch(`/tareas/${id}`, { hecha: true }),

  onMutate: async (id) => {
    await queryClient.cancelQueries({ queryKey: ['tareas'] });
    const previo = queryClient.getQueryData(['tareas']);     // guardar por si falla

    queryClient.setQueryData(['tareas'], (viejas) =>         // pintar YA
      viejas.map(t => (t.id === id ? { ...t, hecha: true } : t)));

    return { previo };
  },

  onError: (_e, _id, ctx) => queryClient.setQueryData(['tareas'], ctx.previo),
  onSettled: () => queryClient.invalidateQueries({ queryKey: ['tareas'] }),
});
```

> Es lo que hace que un check se sienta instantáneo con una red lenta. La parte que casi nadie escribe es `onError`: sin volver atrás, la pantalla miente.

Montarlo en la app

```
// app/_layout.tsx
const queryClient = new QueryClient({
  defaultOptions: { queries: { staleTime: 60_000, retry: 2 } },
});

// En React Native no hay «foco de ventana»: se lo damos nosotros.
AppState.addEventListener('change', (estado) =>
  focusManager.setFocused(estado === 'active'));

export default function Layout() {
  return (
    <QueryClientProvider client={queryClient}>
      <Slot />
    </QueryClientProvider>
  );
}
```

> Sin ese `AppState`, el refresco al volver a la app —una de las razones para usar la librería— no se dispara nunca en móvil.

UNIDAD 5

Documentación

- `developer.mozilla.org/es/docs/Web/API/Fetch_API`
- `axios-http.com/es/`
- `tanstack.com/query/latest`
- `tanstack.com/query/latest/docs/framework/react/react-native`